

Hanson Robotics AI API SDK

Aug 25, 2022

SDK MANUAL

1	System Requirement	3
2	Install Docker	4
2.1	Change Docker Root Dir	4
3	Install AWS CLI	5
4	Install Drivers	6
4.1	PulseAudio	6
4.2	Realsense	6
5	Install SDK Command Tool	7
6	Initialize SDK	8
7	Pull Docker Images	9
8	Run Robot SDK	10
8.1	Start SDK	10
8.2	Run Samples	10
9	Robot Operation	12
9.1	WebUI Page	12
9.2	Performance page	15
9.3	Motor page	18
9.4	Visualization page	19
9.5	Setting page	19
10	Rolling Base	21
11	Command Tool Usage	22
11.1	Initialize SDK	22
11.2	Start SDK	23
11.3	Stop SDK	25
11.4	Pull Images	25
11.5	Login Container Registry	26
11.6	Show Version	26
11.7	Show SDK info	26
12	Install ROS Message Definition Package	27

13 SDK Overview	28
13.1 Services	28
13.2 Full List of Services	29
13.3 ROS API	29
14 Perception	31
14.1 Sensor Access API	31
14.2 Vision API	32
14.3 Perception API	33
15 Interaction	35
15.1 Overview	35
15.2 Interaction API	36
15.3 Behavior API	38
16 Behavior Trees	39
16.1 Behavior Tree ROS API	39
16.2 Behavior Tree Editor	39
17 Control	45
17.1 Actuators API	45
17.2 Audio Control API	49
17.3 Speech Control API	50
17.4 Animation API	52
17.5 Performance API	57
18 Message Definition	60
18.1 Messages	60
18.2 Services	65
19 SDK Troubleshooting	70
20 Robot Troubleshooting	73
20.1 Motor	73
20.2 Perception/Audio	73
21 Indices and tables	75

SYSTEM REQUIREMENT

	System Requirement
OS	Ubuntu 16.04 (amd64) or later LTS
Processor	Quad Core 3.5 GHz or better
Memory	16 GB RAM
GPU	Nvidia GeForce GTX 1050 or better
Network	Broadband Internet connection
Storage	35 GB available space
Other	Ports available on host: 1024, 1025, 1026, 3306, 5555, 5556, 8000, 8103, 8104, 8105, 8106, 8110, 8111, 9002, 9090, 10001, 11311

INSTALL DOCKER

Follow the [official instructions](#) to install Docker.

- For Ubuntu 18.04, there is a [script](#) to accelerate the Docker installation.

After Docker is installed, execute the command below and **sign out and sign in the user or just reboot** so one can run docker without sudo:

```
$ sudo usermod -aG docker $USER
```

If docker is properly functioning the follow shall show the docker info without returning errors:

```
$ docker info
```

2.1 Change Docker Root Dir

Note: This step is optional.

If the disk space is less than **35GB**, it is recommended to mount a larger disk and change the Docker `root dir` to the mounted disk.

The default Docker root dir is `/var/lib/docker`:

```
$ sudo mkdir -p /etc/systemd/system/docker.service.d
$ sudo vi /etc/systemd/system/docker.service.d/docker.root.conf
```

Add the below content to it. Change the dir accordingly:

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -g /new/docker/root -H fd://
```

Then restart docker service:

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart docker
```

Verify the root dir has updated:

```
$ docker info | grep "Root Dir"
```

INSTALL AWS CLI

Run this code:

```
$ sudo apt install python3-pip # not required if pip3 is already installed
$ pip3 install awscli --upgrade --user
$ bash -l # run the login shell in order to call aws command
```

Check the version of AWS CLI. It should be 1.18 or above:

```
$ aws --version

aws-cli/1.18.39 Python/3.6.9 Linux/5.0.0-23-generic botocore/1.15.3
```

INSTALL DRIVERS

4.1 PulseAudio

Install the packages:

```
$ sudo apt-get install libasound2 libasound2-plugins alsa-utils alsa-oss  
$ sudo apt-get install pulseaudio pulseaudio-utils
```

4.2 Realsense

Follow the installation instructions on [librealsense repo](#)

INSTALL SDK COMMAND TOOL

Download the package from *Hanson Robotics* and install it:

```
$ sudo dpkg -i head-hrsdk-tool_x.x.x_amd64.deb
$ sudo apt-get install -f # install any missing dependencies
```

Add bash completion. It is recommended to add the bash completion so it can complete the command when typing TAB on the terminal:

```
$ echo "source <(hrsdk completion bash)" >> ~/.bashrc
$ bash -l # run the login shell in order to use the bash completion
```


INITIALIZE SDK

Note: The required fields can get from the configuration document provided by *Hanson Robotics*

```
$ hrsdk init
```

```
AWS Access Key ID: # get from Hanson Robotics  
AWS Secret Access Key: # get from Hanson Robotics  
Default region name: # get from Hanson Robotics  
SDK license key: # get from Hanson Robotics  
Config name: # get from Hanson Robotics  
SDK Version: # get from Hanson Robotics
```

PULL DOCKER IMAGES

Run the command below to pull the Docker images:

```
$ hrsdk pull
```

If the images exist already, it will update them accordingly.

RUN ROBOT SDK

8.1 Start SDK

The typical way to start the SDK is:

```
$ hrsdk start
```

Then go to the browser and open `http://<IP>:8000`. The IP is the host IP (e.g. `http://127.0.0.1:8000`). The robot teleoperation interface (a.k.a. WebUI) should appear when the SDK is fully up. If it does not open, perhaps the SDK is still starting, so please wait for about one minute or so and refresh the browser.

For more details about robot operation interface check out *Robot Operation*.

For more usage of the `hrsdk` command, refer to *Command Tool Usage*

8.2 Run Samples

Download the samples container:

```
$ docker pull 222132024866.dkr.ecr.ap-east-1.amazonaws.com/hansonrobotics/kinetic_
↳ samples:dev
```

Run the samples container after the pull is finished. It brings one to the bash terminal inside the container.

```
$ docker run --network host -it --rm 222132024866.dkr.ecr.ap-east-1.amazonaws.com/
↳ hansonrobotics/kinetic_samples:dev
```

```
root@hr-desktop:/samples
```

The sample scripts are written in Python. One can choose any of them and run it with Python.

- `actuator_control.py`: An example showing how to control individual actuators.
- `animation_control.py`: An example iterates all the arm animations, facial expressions and head animations.
- `duckduckgo_chat.py`: An example showing how to connect with an online chatbot, DuckDuckGo in this case.
- `echo.py`: An example speaks out what it hears by the connection the STT and the TTS directly.
- `simple_chat.py`: A simple STT to TTS interaction example. It invokes the chat service on the built-in bot.

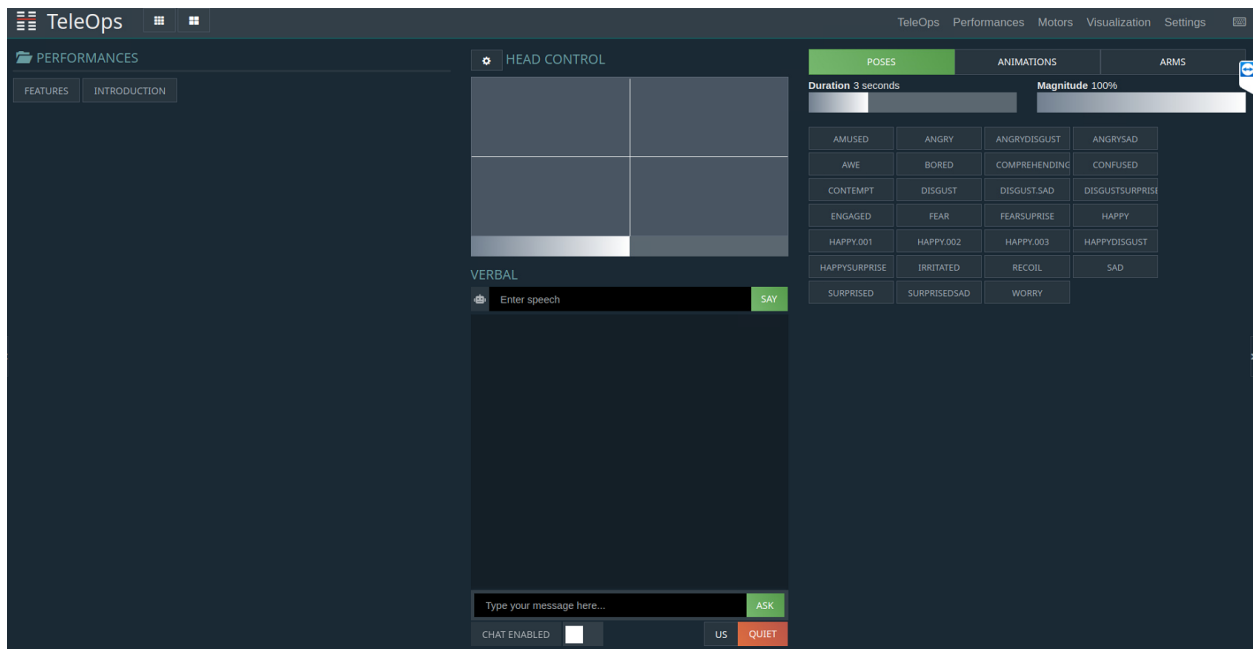
For example, let us run the script called `actuator_control.py`. It periodically sends the actuator commands to move both shoulder rolling actuators. To see what happens on the robot, one can go to the visualization page to see how robot arms move: `http://<IP>:8000/#/virtual`

```
$ python actuator_control.py
```

ROBOT OPERATION

The **teleOperation interface (WebUI)** is the main interface to operate the robot.

After the SDK is launched, launch a browser (e.g. Chrome or IE) on the laptop, and log into TeleOps by entering the address: **<http://10.0.0.10:8000>** if running inside robot. For local SDK go to <http://127.0.0.1:8000>

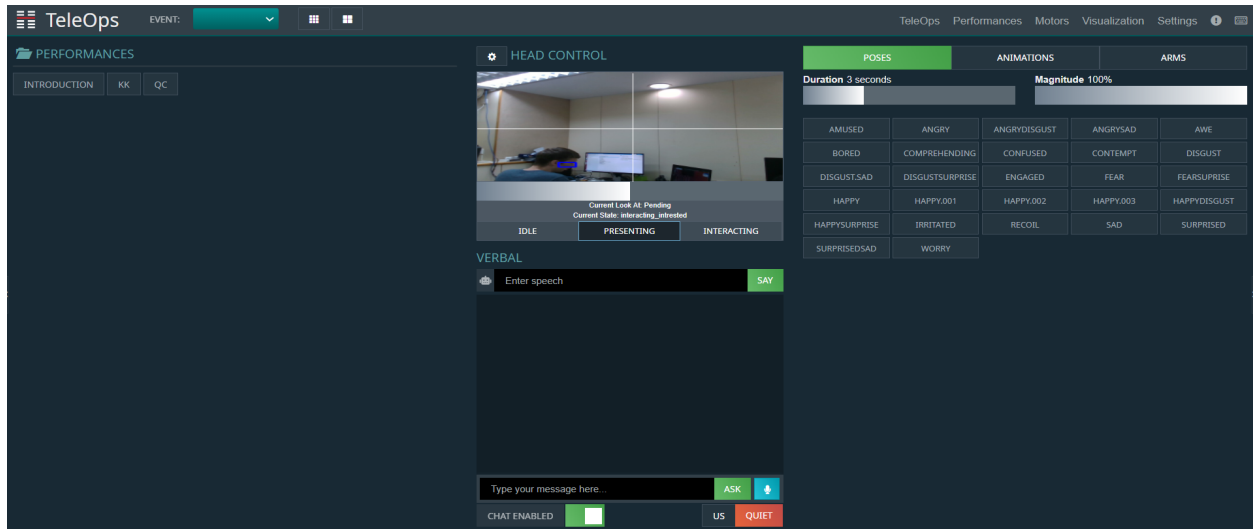


9.1 WebUI Page

At the top is the page menu for TeleOps.

- **TeleOps:** Dashboard.
- **Performances:** Timeline editor to create scripted performances.
- **Motors:** Direct control of the servos.
- **Visualization:** Visualization for the robot of what she doing now.
- **Setting:** Setting of the robot.

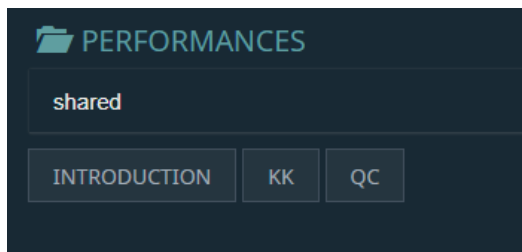
Clicking on the tabs at the top right corner brings up the corresponding page.



TeleOps Dashboard/Main page

The TeleOps Dashboard or Home Page is (vertically) divided into 3 sections. Section 1 is on the left; Section 2 is in the center and Section 3 is on the right side of the screen.

Section 1



This contains a list of performances. The top where the icons have white text are folders that have different performance in them. The bottom where each performance is boxed is only the performance.

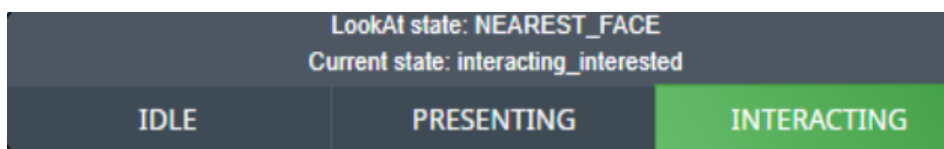


When a performance is clicked, it has the timelines for that performance.

Performance Timelines Selection and Playback:

You can select the timeline to play and control its progression using “Run”, “Auto-Pause” and “Loop” buttons.

Section 2



- Head Control:

By clicking and dragging the cross center on the video screen, it is possible to control Robot's neck and head rotation.

- **LookAt state:**

Listed where the robot is looking at, the setting can be changed in setting page.

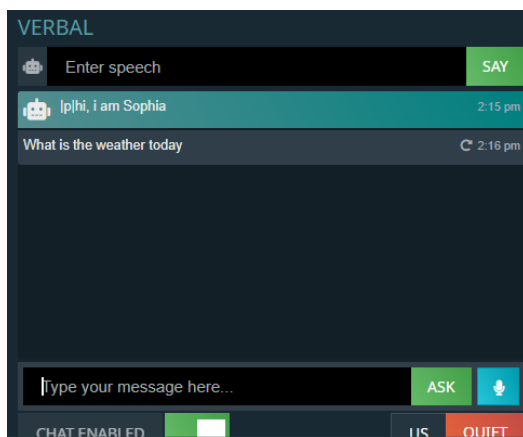
- **Current state:**

The mode of the robot now.

Modes:

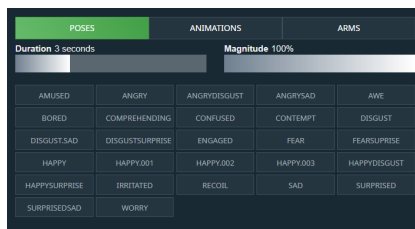
- (i) **Idle:** Basically, the robot would in idle and nothing would be done.
- (ii) **Presenting:** Robot would be fall under this mode when she is in time line performance.
- (iii) **Interactive:** Robot would interactive with others under this mode (e.g. free chat and face tracking).

Operator (Chatbot):

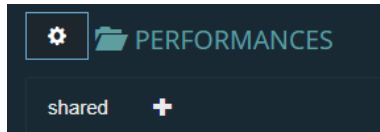


This is the control and monitor screen for the conversations. Chat can be enabled by having the audio input and activate the speech recognition. Chat Enabled means whether the chat is enabled or not

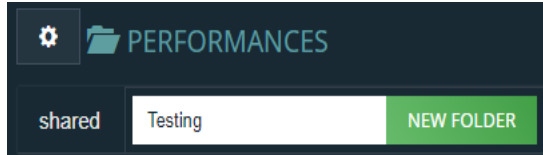
Section 3



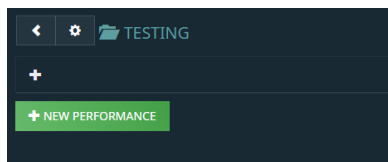
POSES: Allow direct access to the various facial expressions defined for the robot, typically used for pictures or for calibration of facial motors. Duration and Magnitude can be modified using the slider below.



- (b) Fill in the name of the event folder

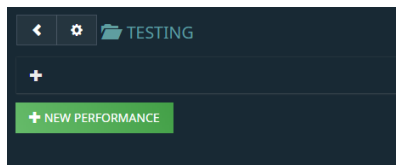


- (c) Click NEW FOLDER button afterward

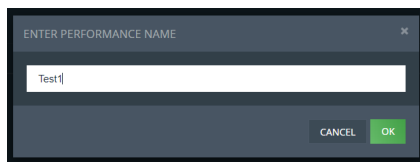


- (2) Create event inside the new event folder

- (a) Click the NEW PERFORMANCE buttons

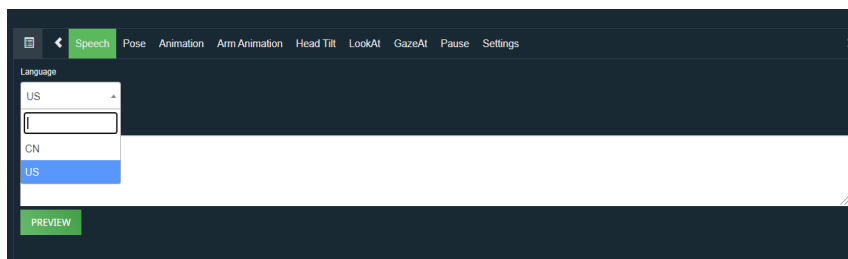


- (b) Enter performance name and press OK

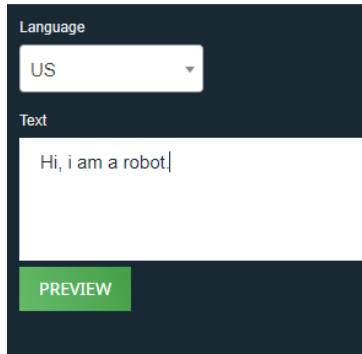


- (3) Insert speech

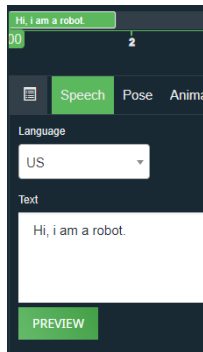
- (a) Select the correct language. CN means chinese and US means English.



- (b) Type the speech in the text box, can be preview by clicking the preview button.

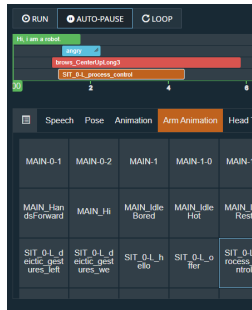


(c) Drag the “Speech” up to the timeline and release to insert the speech.



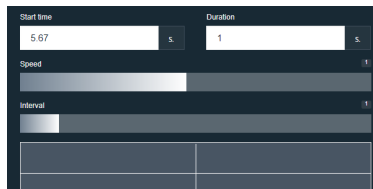
(4) Insert pose/expression/animation

(a) Drag the wanted pose/expression/animation to the timeline to insert corresponding action



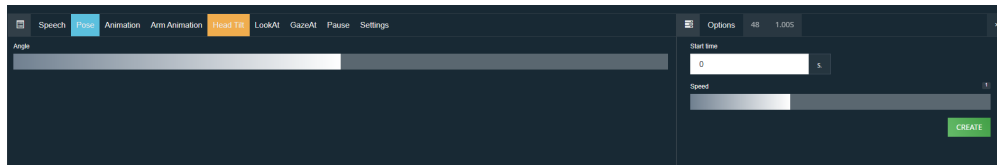
(5) Insert look at/gaze at

(a) Select Look At/Gaze At and select custom, on the right-hand side, there would be a small window on the right, after the look at/gaze at is dragged on the timeline, the start time would auto change. Can use the slider to change corresponding attribute and drag the cross center to where you want the robot to look at.



(6) Insert head tilts

(a) For the head tilt, just use the slider to select the angle wanted and adjust the speed on the right hand side, drag the head tile to the timeline.

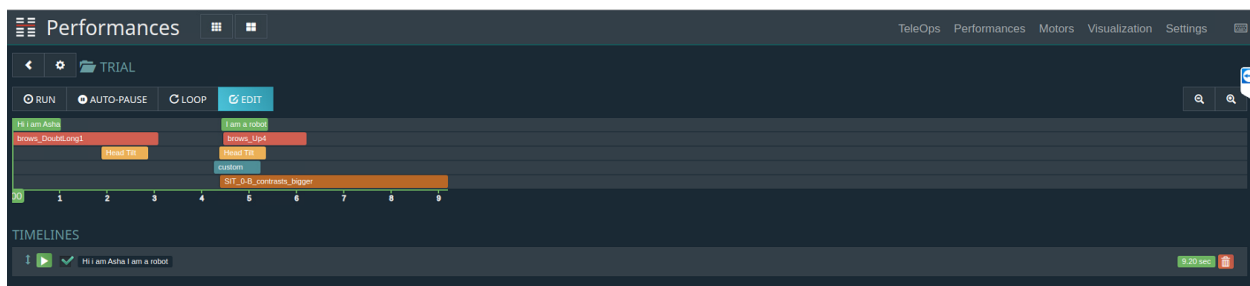


(7) Insert pause (optional)

(8) Insert setting (optional)

Commands (each of the commands below applies only to a single timeline):

- **Run:** to play the timeline
- **Auto-Pause:** to automatically pause the timeline after it ends
- **Loop:** to loop the timeline
- **Edit:** to edit the timeline



9.3 Motor page

(i) Control motor

In motor page, you can control the specific motor by the sliders.



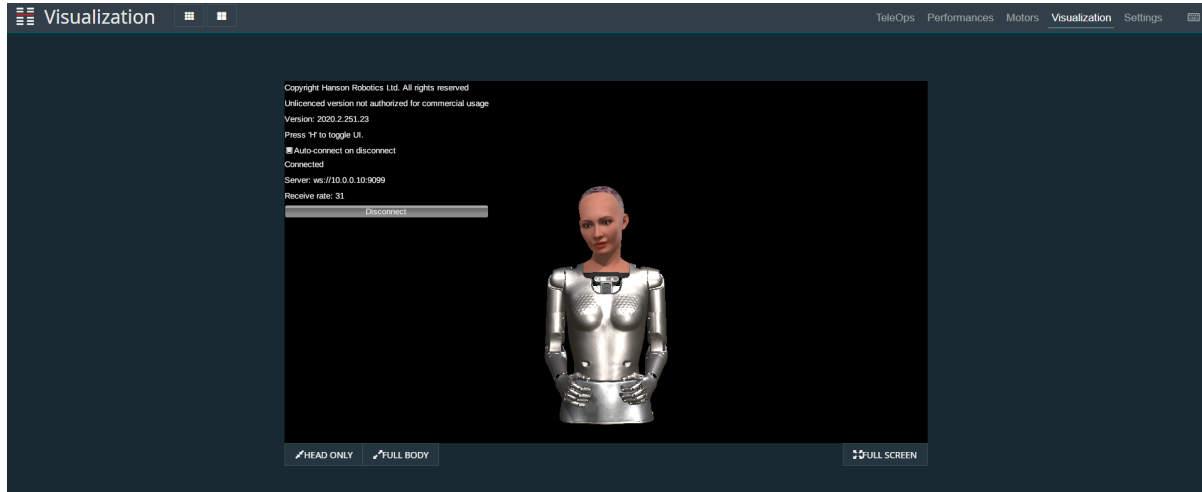
(ii) Motor status

For the dynamixel motor, there would be some data show the motor status, please check the motor status

9.4 Visualization page

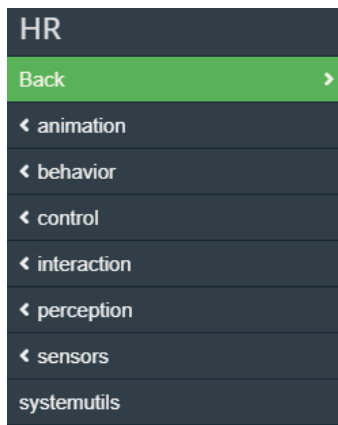
- (i) Image of the robot

In visualization page, you can see what action the robot is doing right now



9.5 Setting page

For the hr node, there are different category under the node:



- (i) Animation

For the animation, it includes animation setting of the robot

- (ii) Behavior

For the behavior, it includes different setting related to the state of the robot

- (iii) Control

For the control, it includes performance generation and tts talker setting of the robot

- (iv) Interaction

For the interaction, it includes the chatbot setting of the robot

(5) Perception

For the perception, it includes the speech recognition and camera setting of the robot

(vi) Sensor

For the sensor, it includes the sensor setting of the robot

(vii) Systemutils

For the systemutils, it include the other setting here

Remarks:

Note: Left/Right Eye camera cannot be used simultaneously.

ROLLING BASE

For rolling robots the external Slamware ROS node is used for controlling the navigation. [Documentation](#)

COMMAND TOOL USAGE

Usage: hrsdk <command> [<args>]

Supported commands:

```
init
    Init Robot SDK
start [options]
    Start Robot SDK
stop
    Stop Robot SDK
pull [options]
    Pull the files from Docker registry and S3
push [options]
    Push the files to Docker registry and S3 [for admin]
completion bash
    Print the bash completion script
login [region]
    Login the ECR with the region
version
    Show the version of this hrsdk tool
info
    Show SDK information
```

The SDK command-line tool is used to initialize, update, and run the SDK.

The following sections will describe the commands in details.

11.1 Initialize SDK

Note: The required fields can get from the configuration document provided by *Hanson Robotics*

```
$ hrsdk init
```

```
AWS Access Key ID: # get from Hanson Robotics
AWS Secret Access Key: # get from Hanson Robotics
Default region name: # get from Hanson Robotics
Registry prefix: # leave it empty or type "default"
```

(continues on next page)

```

SDK license key: # get from Hanson Robotics
Config name: # get from Hanson Robotics
SDK Version: # get from Hanson Robotics

```

After the initialization, one can update the individual SDK settings by running:

```
$ hrsdk init -f
```

If the field does not need to change, type [ENTER] to move to the next field.

11.2 Start SDK

```

hrsdk start [--group] [--service] [--storage <directory>] [--env]
            [--stt] [--gui]
            [--local_ip <IP>] [--master_ip <IP>]
            [--head] [--body]
            [-p] [-c] [-i] [--hw]

--group
    start service group
    use multiple --group options to start multiple service groups
--service
    start the service
    use multiple --service options to start multiple services
--storage <directory>
    the directory for the local storage
--env
    set the environment variable
    use multiple --env options to set multiple environment variables
--stt
    enable google cloud speech
--gui:
    enable UI (for blender)
--local_ip <IP>
    the IP address for the local machine
--master_ip <IP>
    the IP address for the machine runs ROS master
--head
    the name of the robot head
--body
    the name of the robot body
-p:
    start perception container (DEPRECATED)
-c:
    start control container (DEPRECATED)
-i:
    start interaction container (DEPRECATED)
--hw:
    start hardware control container (DEPRECATED)

```

This is perhaps the most important command that is used to start the SDK.

11.2.1 Common Usage

The typical usage is simply `hrsdk start` without any options. It will launch the default service group which contains `control`, `perception`, `interaction`, `control-hw` services.

By default the speech recognition is off after SDK is launched. If one wants to enable speech recognition on start, one can add `--stt` to the command.

11.2.2 Run Service Group

```
$ hrsdk start --group interaction
```

This will start the service group `interaction` (It is equivalent to `-i` option in this case.). One can run multiple service groups by adding multiple `--group` options.

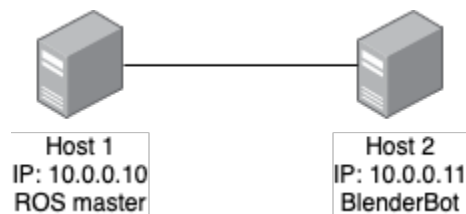
11.2.3 Run Services

One can run an individual service on the machine. For example:

```
$ hrsdk start --service ttsserver
```

This will start a standalone TTS server. One can use `--service` to start a single service. And use it several times to start multiple services. See the [Full List of Services](#)

Because the SDK can run services on multiple machines. It allows to run some services on one machine, and run some other services on another machine. In order to do that, one needs to combine `--local_ip`, `--master_ip`, and `--service`. For example, assume the IP of the host that runs ROS master, and other services is `10.0.0.10`, and we want to run a chat server (e.g. blenderbot) on another machine with the IP `10.0.0.11`, one can run this command on the second machine:



```
$ hrsdk start --service blenderbot --local_ip 10.0.0.11 --master_ip 10.0.0.10
```

11.2.4 Other Usage

The SDK can load the configs from a local directory. The `--storage <directory>` is often used by developers to test the configs on the local disk before committing/publishing the configs.

`--head` and `--body` are used to specify the head and body of the robot if the hardware can not be detected or correctly detected.

`--env` is used to set environmental variables for the SDK. For example, one can pass microphone name by adding `--env MIC_NAME=mic1` that will change the audio perception related topics. One can pass multiple environmental variables by using `--env` multiple times, such as `--env F00=BAR --env F002=BAR2`.

`--gui` option is to show the Blender UI, the animation blending engine. This is often needed in expression calibration.

11.3 Stop SDK

Run the command to stop all the services that the SDK has launched:

```
$ hrsdk stop
```

11.4 Pull Images

```
Usage: hrsdk pull [--reset] [--configs]
      [--images <installation>] [--models]

--reset
    reset the storage
--configs
    pull configs only
--images <service>
    pull the images for the service group
--models
    pull models only
```

Run the command below to pull the Docker images:

```
$ hrsdk pull
```

If the images exist already, it will update them accordingly.

When something goes wrong that the SDK cannot run normally, in order to reset to a factory configuration, one can put a `--reset` flag to the command. Then all the user configs will be reset to the initial settings:

```
$ hrsdk pull --reset
```

Warning: Run this command with caution.

After executing this command, the Docker volume `hrsdk_storage` will get reset. That means any user data stored in this volume will be lost.

If one wants to only update the configs and do not want to update the Docker images, one can put a `--configs` flag to the command:

```
$ hrsdk pull --configs
```

If one wants to only update the images that are related to a specific service, and do not want to update all the Docker images, one can put a `--images` flag to the command:

```
$ hrsdk pull --images <service>
```

11.5 Login Container Registry

Run the command below to login the docker registry and update the authentication token:

```
$ hrsdk login
```

```
Logging ECR
Login ap-east-1 successfully
```

By default it logs in the ap-east-1 region. If the region is a different one, run this command:

```
$ hrsdk login <region>
```

11.6 Show Version

Run this command to show the version of this command tool.

```
$ hrsdk version
```

```
0.3.15
```

Show system-wide information

Run this command to display the system-wide information

```
$ hrsdk info
```

```
SDK Tool Version: 0.3.15
SDK Version: dev
Config name: hr
Registry: 192.168.0.126:30082
```

11.7 Show SDK info

Run this command to show SDK information.

```
$ hrsdk info
```

```
Date: Mon Sep 20 14:55:01 HKT 2021
Host: hr-workstation
OS: Ubuntu 18.04.5 LTS
SDK Tool Version: 0.3.16
SDK Version: dev
Config name: hr
Image digests
    [long image digests omitted]
License user: dev
License expiry: 2022-12-13
AWS Region: ap-east-1
AWS User: hrsdk
```

INSTALL ROS MESSAGE DEFINITION PACKAGE

The ROS message definition package (a.k.a `hr_msgs`) is required for the communication between HRSDK ROS API and external ROS nodes.

The `hr_msgs` is a ROS package that contains the definition of all the message and service types.

One can get a copy of this package from Hanson Robotics and install it:

```
sudo dpkg -i head-<ros-distribution>-hr_msgs_x.x.x_amd64.deb
```

The supported ROS distributions are

- ROS Kinetic:

```
sudo dpkg -i head-kinetic-hr_msgs_x.x.x_amd64.deb
```

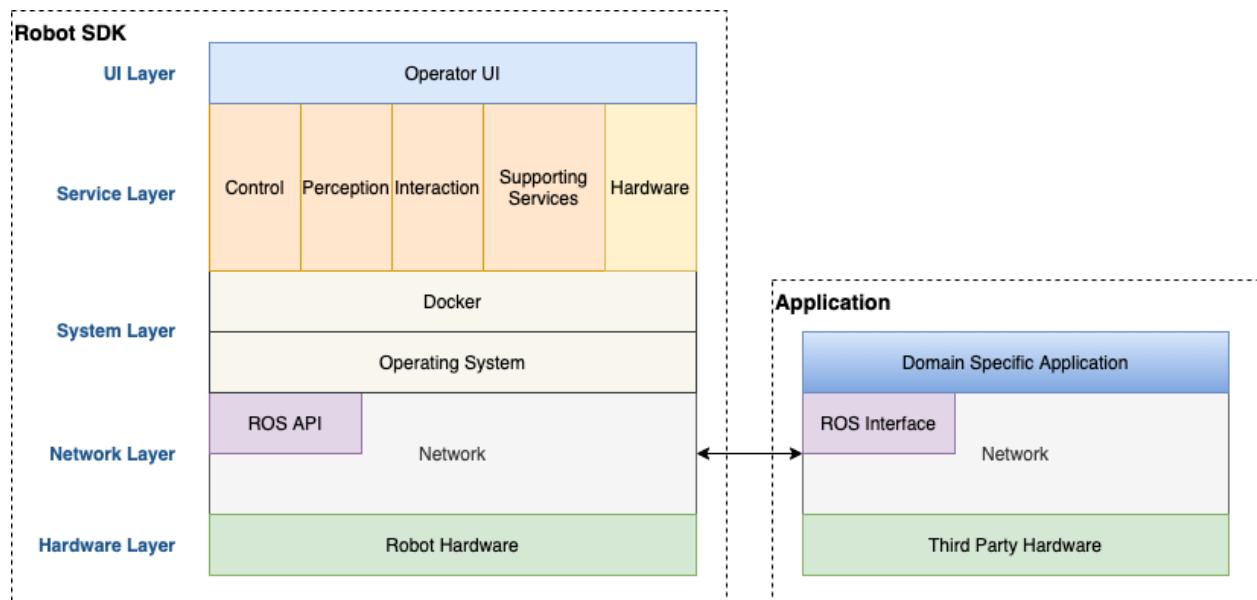
- ROS Melodic:

```
sudo dpkg -i head-melodic-hr_msgs_x.x.x_amd64.deb
```

- ROS Noetic:

```
sudo dpkg -i head-noetic-hr_msgs_x.x.x_amd64.deb
```

SDK OVERVIEW



13.1 Services

The SDK provides the services that work together to operate the robots.

There are 4 major services that run by default:

1. Control (`control`) service: provides the control APIs that control the robot what to say, how to move etc. It also contains a web interface for robot manipulation, and visualization.
2. Interaction (`interaction`) service: provides the interaction APIs. It has a built-in chatbot service that can have a chitchat with people. Developers can also integrate their own chatbot using the provide ROS chat API.
3. Perception (`perception`) service: provides the perception APIs. It streams the raw audio from the mic and video from the cameras. And it runs the speech recognition.
4. Hardware (`control-hw`) service: provides the hardware level of control for the actuators, sensors

13.2 Full List of Services

1. `asr`: Automatic speech recognition service
2. `bert`: BERT embedding service
3. `blenderbot`: BlenderBot chat service
4. `chatbot-server`: Chatbot REST server
5. `chatscript`: ChatScript chat service
6. `control-hw`: Robot hardware-level control service
7. `control`: Robot high-level control service
8. `duckling`: A Haskell library that parses text into structured data
9. `gpt2-server`: The GPT-2 service as a chat agent
10. `home`: A volume directory sharing service
11. `interaction`: Interaction service
12. `mysql`: Database service
13. `navigation`: The rolling base navigation service
14. `neo4j`: The Neo4J graph database
15. `nluserver`: NLU service
16. `openface`: The OpenFace service that does face action unit detection
17. `perception-advance`: Advance facial perception service
18. `perception`: audio & vision perception service
19. `rosmaster`: ROS master node
20. `sensors`: The sensors service that provides all the raw video/audio sensor input.
21. `soultalk`: SOULTalk chat service
22. `storage`: Storage service for SDK configs synchronization
23. `ttsserver`: TTS service

13.3 ROS API

Mainly all APIs are set to 4 categories: ROS topics, ROS services, ROS parameters, ROS dynamic reconfigure.

13.3.1 ROS Topics

By default all Topics services parameters are under `/hr` namespace.

13.3.2 ROS Services

ROS Services are another means of communication among nodes. They follow the request/response paradigm that allow a two-way interaction.

A node makes a service call by sending the message to the node that provides this service and gets the response from the service provider.

13.3.3 ROS Parameters

The ROS parameters stores the settings for the various ROS nodes running in the services. They are all loaded to ROS master on startup for setting up the ROS nodes.

13.3.4 ROS Dynamic Reconfigure

Most nodes use dynamic reconfigure to change the behavior settings during runtime without having to restart the node.

Important: Additional topics/services/parameters that are available should not be used, as they are for communication between internal components.

PERCEPTION

Raw perception includes the raw audio/video data, and some other sensory data such as imu and proximity

14.1 Sensor Access API

The raw APIs provide the raw sensory data.

Each robot hand is equipped with the 11 sensors: 5 finger tip touch sensors and 6 potentiometers in 4 fingers and 2 potentiometers for wrist.

Touch sensors are calibrated so the 0 or close to 0 value is non-touching and increase in value will indicate force applied on finger tips.

Potentiometers are still an experimental future and they provide raw values in different ranges.

14.1.1 Topics Published

/hr/sensors/vision/realsense/camera/accel/sample

Message Type *sensor_msgs/Imu*

Description Linear Acceleration

/hr/sensors/vision/realsense/camera/gyro/sample

Message Type *sensor_msgs/Imu*

Description Angular Velocity

/hr/sensors/left_hand

Message Type *hr_msgs/TargetPosture*

Description Left hand pressure sensors and potentiometers

/hr/sensors/right_hand

Message Type *hr_msgs/TargetPosture*

Description Right hand pressure sensors and potentiometers

/hr/sensors/audio/<microphone name>

Message Type *hr_msgs/AudioStream*

Description The audio stream from the microphone. The default microphone name is `speech_recognizer`

14.1.2 List of Hand Sensors

Topic API	Description
RightThumbPress	Pressure sensor
RightIndexPress	Pressure sensor
RightMiddlePress	Pressure sensor
RightRingPress	Pressure sensor
RightPinkyPress	Pressure sensor
RightThumbAngle	Potentiometer
RightIndexAngle	Potentiometer
RightMiddleAngle	Potentiometer
RightRingAngle	Potentiometer
RightWristPitchAngle	Potentiometer
RightWristRollAngle	Potentiometer
LeftThumbPress	Pressure sensor
LeftIndexPress	Pressure sensor
LeftMiddlePress	Pressure sensor
LeftRingPress	Pressure sensor
LeftPinkyPress	Pressure sensor
LeftThumbAngle	Potentiometer
LeftIndexAngle	Potentiometer
LeftMiddleAngle	Potentiometer
LeftRingAngle	Potentiometer
LeftWristPitchAngle	Potentiometer
LeftWristRollAngle	Potentiometer

14.2 Vision API

14.2.1 Topics Published

/hr/sensors/vision/realsense/camera/color/image_raw

Message Type `sensor_msgs/Image`

Description RGB image stream from realsense

/hr/sensors/vision/realsense/camera/aligned_depth_to_color/image_raw

Message Type `sensor_msgs/Image`

Description Aligned depth to RGB stream from realsense

/hr/sensors/vision/realsense/camera/aligned_depth_to_infra1/image_raw

Message Type `sensor_msgs/Image`

Description Aligned depth to infra stream from realsense

/hr/sensors/vision/chest_cam/usb_cam/image_raw

Message Type `sensor_msgs/Image`

Description RGB stream from chest camera

/hr/sensors/vision/eye_cam/usb_cam/image_raw

Message Type `sensor_msgs/Image`

Description RGB stream from eye camera

14.3 Perception API

14.3.1 Topics Published

/hr/perception/hear/event

Message Type `std_msgs/String`

Description Value can be speechstart, speechstop, or speechcont.

speechstart: user starts to speak

speechstop: user stops to speak

speechcont: user speech continues

/hr/perception/hear/sentence

Message Type `hr_msgs/ChatMessage`

Description Receives the heard final phrases

/hr/perception/hear/words

Message Type `hr_msgs/ChatMessage`

Description Heard words

/hr/perception/hear/interim_speech

Message Type `hr_msgs/ChatMessage`

Description Interim results from speech recognition

/hr/perception/vad

Message Type `hr_msgs/VADMessage`

Description Each message is a Voice Activity Detection signal with some information about the frequency, decibel and duration.

/hr/perception/face_markers

Message Type `visualization_msgs/Marker` <http://docs.ros.org/api/visualization_msgs/html/msg/Marker.html>

/hr/perception/ui_image

Message Type `sensor_msgs/Image`

Description Image of what the robot's camera sees

/hr/perception/hand_events

/hr/perception/joint_states

/hr/perception/poses

14.3.2 Topics Subscribed

/hr/perception/state

Message Type *hr_msgs/State*

Description Details and important features of what the robot's camera sees

14.3.3 Dynamic Reconfigure

/hr/perception/<microphone name>

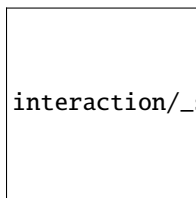
The default microphone name is `speech_recognizer`.

Parameters

- **continuous** (*bool*)
Enable Continuous Listening
- **enable** (*bool*)
Enable Speech Recognition
- **init_phrases** (*String*)
Set initial phrases
- **language** (*String*)
Speech Recognition Language e.g.: en-US
- **phrases** (*String*)
Set fixed phrases
- **punctuation** (*bool*)
Enable Automatic Punctuation (if available)

INTERACTION

15.1 Overview



interaction/_static/sdk/interaction-module-diagram0420.png

The core of the interaction module consists of an ensemble of multiple chat agents, and a ranking model. It also contains a NLU service, and a logging system for the chat logs. To make it useful for the Hanson robots, it also contains the ROS interface.

15.1.1 Agents

The chat agents can be

- Open web services such as DuckDuckGo, OpenWeatherAPI, WolframAlpha etc
- Public/private knowledge base query agent such as WikiData
- Third party dialog services such as Xiao-i, Baidu Unit
- Rule-based such as ChatScript, AIML
- Hybrid of rule-based and ML models such as SoulTalk
- Generative models such as GPT-X
- Open domain transformer models such as BlenderBot
- Human supervisor

15.1.2 Ranking Model

What the ranking model does is to rank the responses from different chat agents based on the dialog state, or the hand-crafted rules, or semantics, or other measures. The ranking model helps the dialog system to choose the best response based on the scores. The ranking model should be designed to be flexible to adapt to new chat agents and work with different combinations of chat agents.

15.1.3 Logging System

The logging system logs all the details of the dialog system during the entire conversation in accordance with privacy regulations and laws. The data recorded in the hybrid mode will be used in the debugging of the dialog system, and to serve as a training corpora for training or fine-tuning ML models such as ranking models.

15.2 Interaction API

15.2.1 Topics Published

/hr/interaction/chatbot_input

Message Type *hr_msgs/ChatMessage*

Description Input provided to chatbot

/hr/interaction/chatbot_response

Message Type *hr_msgs/ChatResponse*

Description Single response provided by chatbot

/hr/interaction/chatbot_responses

Message Type *hr_msgs/ChatResponses*

Description Array of responses provided by chatbot multiple agents

15.2.2 Services

/hr/interaction/available_agents

Message Type *hr_msgs/StringArray*

Description List the available chat agents

/hr/interaction/available_forms

Message Type *hr_msgs/StringArray*

Description List the available forms

/hr/interaction/available_scenes

Message Type *hr_msgs/StringArray*

Description List the available scenes

/hr/interaction/chat

Message Type *hr_msgs/ChatService*

Description The chat service

/hr/interaction/get_context

Message Type *hr_msgs/StringTrigger*

Description Get the context from the chat agents

/hr/interaction/get_intent

Message Type *hr_msgs/GetIntent*

Description Get the intent for the text

/hr/interaction/session

Message Type *hr_msgs/ChatSession*

Description The session for the conversation between user and chatbot

/hr/interaction/set_scene

Message Type *hr_msgs/SetScene*

Description Set the scene for the chatbot

/hr/interaction/register

Message Type *hr_msgs/AgentRegister*

Description Register the ROS node as a chat agent

/hr/interaction/unregister

Message Type *hr_msgs/AgentUnregister*

Description Unregister the ROS node as a chat agent

/hr/interaction/run_form

Message Type *hr_msgs/StringTrigger*

Description Trigger the form

15.2.3 Dynamic Reconfigure

/hr/interaction/chatbot

Parameters

- **enable** (*bool*)
Enable chatbot
- **hybrid_mode** (*bool*)
Enable hybrid mode
- **enable_wake_phrase** (*bool*)
Enable wake phrases. The wake phrases are a set of keyphrases to activate the autonomous interaction. It will be deactivated when idle. See the next parameter.
- **hybrid_when_idle** (*int*)
Go to hybrid when idle
- **listen_speech** (*bool*)

-
- Listen to speech
 - **scene** (*string*)
Set scene
 - **fast_score** (*int*)
The minimum required response score, the smaller the faster
 - **timeout** (*float*)
Chatbot Timeout
 - **chatscript_enabled** (*bool*)
Enable ChatScript (English chat agent)
 - **blenderbot_enabled** (*bool*)
Enable BlenderBot (English chat agent)
 - **gpt2_enabled** (*bool*)
Enable GPT2 (English chat agent)
 - **tgbot_enabled** (*bool*)
Enable Telegram Bot (English chat agent)
 - **aimlbot_enabled** (*bool*)
Enable AIML Bot (English chat agent)
 - **soultalk_enabled** (*bool*)
Enable SoulTalk (English chat agent)
 - **ddg_enabled** (*bool*)
Enable DuckDuckGo (English chat agent)
 - **xiaoi_enabled** (*bool*)
Enable Xiao-i (Chinese chat agent)
 - **baidu_unit_enabled** (*bool*)
Enable Baidu Unit (Chinese chat agent)
 - **translator_enabled** (*bool*)
Enable Translator chat agent

15.3 Behavior API

BEHAVIOR TREES

Warning: Behavior tree editor is still experimental feature, and may contain some unexpected behaviors and not consistent UI experience

16.1 Behavior Tree ROS API

16.1.1 Services

/hr/behavior/interactive_fiction/start_tree

Message Type *hr_msgs/RunByName*

Description Run behavior tree by specified title inside behavior tree description.

/hr/behavior/interactive_fiction/cancel_tree

Message Type *std_srvs/Trigger*

Description Cancel the running behavior tree

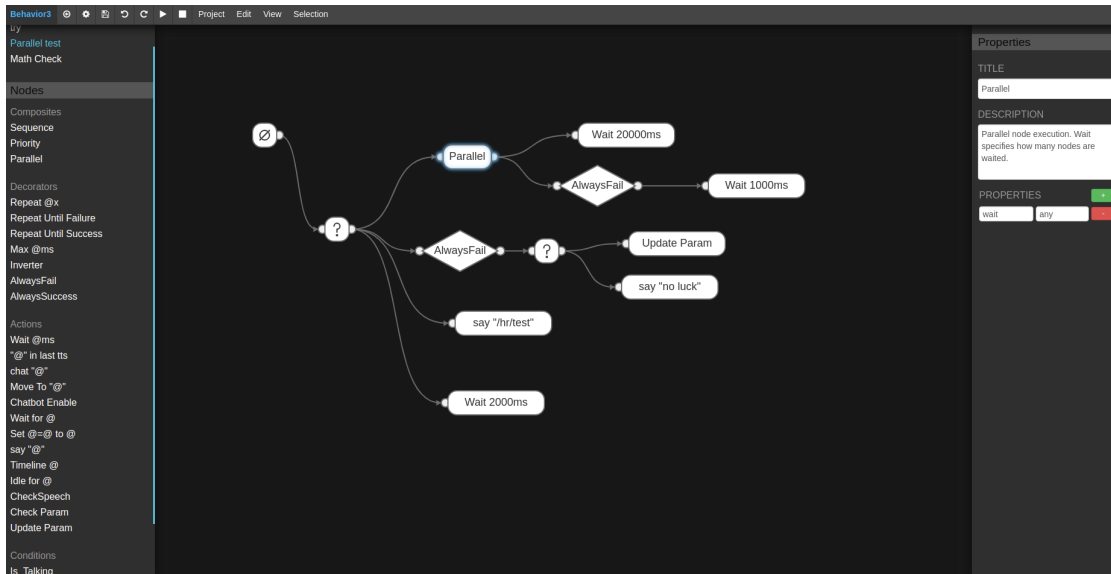
/hr/behavior/interactive_fiction/reload_trees

Message Type *std_srvs/Trigger*

Description Reloads trees from a file. Used by UI after save.

16.2 Behavior Tree Editor

Behavior tree authoring tool mainly used to make new behaviors or adjust existing. It follows standard behavior tree principles. However because implementation done in the asynchronous framework so nodes only return SUCCESS and FAILURE, while longer operations and parent nodes could be run in parallel and if needed be cancelled.



The behavior tree utilizes only other HRSDK topics and services. HRSDK behavior trees integrates tightly with ROS parameters and could be used instead of backboard.

ROS params that are passed as node params would resolve at runtime, at could be referred as:

1. `/hr/parameter_path/parameter_name`: Absolute parameters inside `/hr` namespace.
2. `/parameter_name`: All other values starting with `/` would be treated as relative parameters to `/hr/behavior/interactiv_fiction/backboard` namespace.

Important: Be aware that correct parameter types should be used to avoid unexpected behaviors

Note: Boolean value could be referred by 0 for False and 1 for True

16.2.1 Supported nodes:

Composites

Composite nodes executes multiple children.

Sequence

Executes children in order until first FAILURE. Returns SUCCESS if all children succeed. FAILURE otherwise

Priority

Also known as Selector, executes all children in order until first SUCCESS. Returns FAILURE if all children fails. SUCCESS otherwise.

Decision

If condition is met (1st child), evaluate and return outcome of success_tree (2nd child). If condition not met, either return FAILURE (if only 2 children exist), or evaluate and return outcome of failure_tree (if 3rd child exists). Returns ValueError if number of children <2 or >3.

Parallel

Wait *all* waits for all nodes to finish. *any* waits for first node.

Combine if 'wait=all': 'or' returns SUCCESS if 1 node succeeds. 'and' returns SUCCESS if all finished nodes succeed.

Note: Running some nodes in parallel may lead to unexpected behaviors.

Decorators

Decorator only have one child node.

Repeat

MaxLoop times to repeat

Repeats childr *maxLoop* times. Returns SUCCESS after.

Repeat Until Failure

MaxLoop maximum retry times

SUCCESS if childr failed. FAILURE if 'maxLoop' times executed without FAILURE

Repeat Until Success

MaxLoop maximum retry times

SUCCESS if child succeed. FAILURE if retried 'maxLoop' times without succeed.

Max time (ms)

MaxTime maximum allowed time for child execution in ms.

Returns FAILURE if child did not finish in given time. Otherwise returns child value.

Inverter

Returns opposite result than child.

AlwaysFail

Returns FAILURE regardless of child.

AlwaysSuccess

Returns SUCCESS regardless of child.

Actions and Conditions

Action and Conditions implementation is basically same as there are no RUNNING state. Action and condition cant have child nodes.

Wait (ms)

Milliseconds time in ms.

Waits for specified time and Return SUCCESS

In Last Heard Sentence

Text text to match

Returns SUCCESS if *text* is within the last sentence it heard. Fail otherwise.

Chat

Text text to send

Language language that text is in

Sends given *text* to chatbot

Move To

Location Predefined Location

Starts navigating robot to predefined location. Immediately return SUCCESS.

Chatbot Enable

Enabled 1 - enable, 0 - disable

Returns SUCCESS if service call succeeded. FAILURE on error.

Check Time

From min. time boundary; current time should be after this to return TRUE

To max. time boundary; current time should be before this to return TRUE

Returns SUCCESS if current time is in between times specified in parameters

Trigger Event

Event event name

Triggers some event; returns SUCCESS when triggered

Wait For

Event event names. Following events are supported:

- *speech*: Speech to text returned result
- *vad*: Voice activity detection
- *tts_start*: robot start speaking
- *tts_end*: robot stop speaking
- *timeline start*: Timeline performance started
- *timeline stopped*: Timeline performance stopped or paused.

Timeout time to wait.

Returns SUCCESS if event happened. FAILURE if timeout was reached.

Set Dynamic Param

Node node ROS absolute name.

Param param name.

Val param value

Sets dynamic reconfigure parameter to node. Returns SUCCESS if set. FAILURE on error.

Set Dynamic Params

Node node ROS absolute name.

Params dictionary containing param name-value pairs.

Sets dynamic reconfigure parameters to node. Returns SUCCESS if set. FAILURE on error.

Say

Text Text to say

Language language that TTS message is in

Wait_for_stop 1 - wait until tts finished. 0 - return immediately

Sends TTS message. Waits for TTS to finish if *wait_for_stop* is set.

Animation

Name name of animation

Magnitude magnitude of animation, default is 1.0

Speed speed of animation, default is 1.0

Produces animation based on parameters.

Timeline

Timeline timeline number combined with performance_name and parameter_path. e.g”
welcome/1

Wait_for_stop 1 - wait until timeline finished. 0 - Return immediately

If only performance path and name ios given the random timeline would be played. If *wait_for_stop* is set, will wait until performance is stopped.

Idle

Min_time min time for robot to be idle

Wait_for_idle 1 - wait for idle, 0 - check for idle

Checks if robot is idle at last *min_time*. if *wait_for_idle* is set waits until given condition.

CheckSpeech

Keywords Comma separated keywords to be matched

Intent intent to be matched. Current supported intents: *yes, no*.

Wait_for_input 1 - waits for new speech input. 0 - uses last known speech input.

No_match_retry 1 - retry the speech input if it's new speech input and it wasn't matched.

Match_retry 1 - retry to chatbot if new speech input was matched

Matches speech input to given keywords ior intent. Disables chatbot if waiting for new input. Re-enables chatbot after input received and forwarded again to chatbot based on *retry* flags.

Note: Not recommended to run few such nodes in parallel

Check param

Param Param name

Operation comparison operator

Value comparison value

Checks ROS param based on given comparison operation. Returns FAILURE on error Current supported operations:

- All: *equals*
- Strings: *contains, startswith, endswith*

-
- Numbers: *lt*, *lte*, *gt*, *gte*, *range* (less than, less than or equal greater than, greater than or equal).

Note: For range operator value should be comma separated two numbers.

Update Param

Param Param name

Operation comparison operator

Value comparison value

Param_type Specify parameter type explicitly. *auto* by default.

Updates ROS parameter. Returns FAILURE on error Assigns the new value to param or updates the existing value with given operation and value. Current supported operations:

- All: *assign*
- Strings: *prepend*, *append*
- Numbers: *add*, *multiply*, *min*, *max*, *mod* (less than, less than or equal greater than, greater than or equal).

Note: Make sure parameter is set before applying modification operations.

Person

Uid 'any' (default if nothing entered) identifies all people and if any person fulfills parameters, returns SUCCESS. If not 'any', identifies one person

Distance max. distance person should be from robot

Recognized whether face should be recognized

Unrecognized whether face should be unrecognized

Smiling whether face should be smiling

Eyes_closed whether eyes should be closed

Min_angle min angle of face from camera

Max_angle max angle of face from camera

Keeps track of person(s). Based on whether parameters are fulfilled, returns SUCCESS. Otherwise return FAILURE.

Is Talking

Return SUCCESS if robot is talking.

Is Listening

Returns SUCCESS if someone is currently saying something to robot.

People Detected

Returns SUCCESS if people visible within last second

CONTROL

17.1 Actuators API

Actuators controlled by direct control or by animation data. In this section the direct control is described. For animation data please see the Animation API.

17.1.1 Topics Subscribed

/hr/actuators/pose

Message Type *hr_msgs/TargetPosture*

Description Sets pose for one or multiple actuators (position control) * published by actuators

17.1.2 Topics Published

/hr/actuators/dynamixel/motor_states

Message Type *hr_msgs/MotorStateList*

Description Provides feedback from actuators (where available, dynamixel motors only, others provide last sent position)

/hr/actuators/lhand/pose

Message Type *hr_msgs/TargetPosture*

Description Gets pose of actuators of left hand. The actuators contain

- LeftFingerSpread
- LeftIndexFinger
- LeftMiddleFinger
- LeftPinkyFinger
- LeftRingFinger
- LeftThumbFinger
- LeftThumbRoll

/hr/actuators/rhand/pose

Message Type *hr_msgs/TargetPosture*

Description Gets pose of actuators of right hand. The actuators contain

-
- RightFingerSpread
 - RightIndexFinger
 - RightMiddleFinger
 - RightPinkyFinger
 - RightRingFinger
 - RightThumbFinger
 - RightThumbRoll

/hr/actuators/outer/pose

Message Type *hr_msgs/TargetPosture*

Description Gets pose of outer actuators. The actuators contain

- BrowCenter
- BrowInnerLeft
- BrowInnerRight
- BrowOuterLeft
- BrowOuterRight
- CheekSquintLeft
- CheekSquintRight
- EyesUpDown
- FrownLeft
- FrownRight
- LipsStretchLeft
- LipsStretchRight
- LowerLipCenter
- LowerLipLeft
- LowerLipRight
- SneerLeft
- SneerRight
- UpperLipCenter
- UpperLipLeft
- UpperLipRight

/hr/actuators/top/pose

Message Type *hr_msgs/TargetPosture*

Description Gets pose of top actuators. The actuators contain

- EyeTurnLeft
- EyeTurnRight
- LowerLidLeft

- LowerLidRight
- UpperLidLeft
- UpperLidRight

17.1.3 Services

/hr/actuators/set_control

Message Type *hr_msgs/SetActuatorControl*

Description Set motor control to animation, manual or disabled.

/hr/actuators/dynamixel/set_torque_enable

Message Type *hr_msgs/MotorValuesBool*

Description Switch torque on/off for dynamixel ac

/hr/actuators/dynamixel/set_torque_limit

Message Type *hr_msgs/MotorValues*

Description Switch torque on/off for dynamixel actuators

/hr/actuators/dynamixel/set_actuator_speeds

Message Type *hr_msgs/MotorValues*

Description Limit speed for specific motors

17.1.4 Parameters

Parameter Name	Description
/hr/motors	actuator parameters: {name}/min: minimum angle of joint/actuator {name}/max: maximum angle of joint/actuator

17.1.5 Actuator List

Actuator Name	Model	Gear Ratios
BrowOuterLeft	Xpert RC Micro CI-240-HV	
BrowInnerLeft	Xpert RC Micro CI-240-HV	
BrowCenter	Xpert RC Micro CI-240-HV	
BrowInnerRight	Xpert RC Micro CI-240-HV	
BrowOuterRight	Xpert RC Micro CI-240-HV	
UpperLidLeft	Xpert RC Micro CI-240-HV	
UpperLidRight	Xpert RC Micro CI-240-HV	
LowerLidLeft	MKS DS65K	
LowerLidRight	MKS DS65K	
EyeTurnLeft	Xpert RC Micro CI-240-HV	
EyeTurnRight	Xpert RC Micro CI-240-HV	

continues on next page

Table 1 – continued from previous page

Actuator Name	Model	Gear Ratios
EyesUpDown	MKS HBL-6625MINI	
CheekSquintLeft	MKS HBL-6625MINI	
CheekSquintRight	Xpert RC Micro CI-240-HV	
UpperLipLeft	Xpert RC Micro CI-240-HV	
UpperLipCenter	Xpert RC Micro CI-240-HV	
UpperLipRight	Xpert RC Micro CI-240-HV	
LowerLipLeft	Xpert RC Micro CI-240-HV	
LowerLipCenter	Xpert RC Micro CI-240-HV	
LowerLipRight	Xpert RC Micro CI-240-HV	
LipsStretchLeft	Xpert RC Micro CI-240-HV	
FrownLeft	Xpert RC Micro CI-240-HV	
FrownRight	Xpert RC Micro CI-240-HV	
SmileLeft	Dynamixel XM430 -W350-R	
SmileRight	Dynamixel XM430 -W350-R	
Jaw	Dynamixel XM430 -W350-R	
Tongue*	Xpert RC Micro CI-240-HV	
UpperGimbalLeft*	Dynamixel XM430 -W350-R	
UpperGimbalRight*	Dynamixel XM430 -W350-R	
LowerGimbalLeft*	Dynamixel XM430 -W350-R	
LowerGimbalRight*	Dynamixel XM430 -W350-R	1:1
NeckRotation*	Dynamixel XM430 -W350-R	1:2
RightShoulderPitch	Dynamixel MX106	1:2
RightShoulderRoll	Dynamixel MX106	1:2
RightShoulderYaw	Dynamixel XM430 -W350-R	1:2.916
RightElbowPitch	Dynamixel MX64	1:2
RightElbowYaw	Dynamixel XM430 -W350-R	1:1.8
RightWristPitch	Dynamixel XM430 -W350-R	1:1.47
RightWristRoll	Dynamixel XM430 -W350-R	1:1.33
RightThumbRoll*	Xpert RC Micro CI-240-HV	
RightThumbFinger	Xpert RC Micro CI-240-HV	
RightFingerSpread	MKS HV 75K	
RightIndexFinger	Xpert RC Micro CI-240-HV	
RightMiddleFinger	Xpert RC Micro CI-240-HV	
RightRingFinger	Xpert RC Micro CI-240-HV	
RightPinkyFinger	Xpert RC Micro CI-240-HV	
LeftShoulderPitch	Dynamixel MX106	
LeftShoulderRoll	Dynamixel MX106	1:2
LeftShoulderYaw	Dynamixel XM430 -W350-R	1:2
LeftElbowPitch	Dynamixel MX64	1:2.916
LeftElbowYaw	Dynamixel XM430 -W350-R	1:1.8
LeftWristPitch	Dynamixel XM430 -W350-R	1:1.47
LeftWristRoll	Dynamixel XM430 -W350-R	1:1.33
LeftThumbRoll*	Xpert RC Micro CI-240-HV	
LeftThumbFinger	Xpert RC Micro CI-240-HV	
LeftFingerSpread	MKS HV 75K	
LeftIndexFinger	Xpert RC Micro CI-240-HV	
LeftMiddleFinger	Xpert RC Micro CI-240-HV	
LeftRingFinger	Xpert RC Micro CI-240-HV	
LeftPinkyFinger	Xpert RC Micro CI-240-HV	

continues on next page

Table 1 – continued from previous page

Actuator Name	Model	Gear Ratios
Base*	Dynamixel MX106	1:1

* No Visualization

17.1.6 Dynamic Reconfigure

/hr/control/arms

Parameters

- **arms_mode** (*string*)
Options: manual, off, torque_off, animations
- **arms_pose** (*string*)
Set desired arms pose e.g.: 'sitting'

/hr/control/performance_generation

Parameters

- **lines** (*string*)
Lines
- **name** (*string*)
Performance path
- **pause_between_sentences** (*float*)
Set pause time between pause between sentences
- **node_schema** (*string*)
Set the node schema
- **start** (*bool*)
Start Generation?

17.2 Audio Control API

17.2.1 Topics Subscribed

/hr/control/audio/stream

Message Type `std_msgs/UInt8MultiArray`

Description The audio data sent to this topic will be played on the robot

17.3 Speech Control API

17.3.1 Topics Subscribed

/hr/control/speech/say

Message Type *hr_msgs/TTS*

Description Tell the robot what to say

/hr/control/speech/tts_control

Message Type *std_msgs/String*

Description

- Send the string `shutup` to this topic to interrupt the TTS
- Send the string `ready` to this topic tell the TTS node that it is ready to execute. Used in the animation generation which is done out of the TTS.

/hr/control/speech/tts_stream

Message Type *std_msgs/String*

Description Path and filename of audio file passed to tts stream.

17.3.2 Topics Published

/hr/control/speech/event

Message Type *std_msgs/String*

Description The speech event indicates that the robot is speaking. It can be a sequence of `start`, `stop`, and `duration:<time>`. The time is in seconds indicates the duration of the speech.

/hr/control/speech/data (topic)

Message Type *std_msgs/String*

Description The exact breakdown into words, variables and times spoken for each component of a single instance of talking. This is a string in JSON format.

17.3.3 Services

/hr/control/speech/length

Message Type *hr_msgs/TTSLength*

Description Length of the TTS audio in seconds.

/hr/control/speech/data (srv)

Message Type *hr_msgs/TTSData*

Description Responds with speech when requested with text and language code

/hr/control/speech/interrupt

Message Type *hr_msgs/StringTrigger*

Description Interrupts TTS execution.

The service accepts a string as an interruption type: `soft` | `hard`

`soft` interrupt stops the TTS at the nearest punctuation marks.

`hard` interrupt stops the TTS at the nearest next word.

17.3.4 Dynamic Reconfigure

`/hr/control/speech/tts_talker`

Parameters

- **enable** (*bool*)
Enable TTS
- **lipsync_enabled** (*bool*)
Lipsync Enabled?
- **lipsync_blender** (*bool*)
Do Blender Lipsync?
- **execute_marker** (*bool*)
Execute markers in text?
- **ignore_repeat** (*bool*)
Do not run TTS for the same text as the previous one when in chat
- **tts_delay** (*float*)
Delay the audio for a specific time (in seconds)
- **wait_for_tts_ready** (*bool*)
Wait for TTS to send `ready` message?
- **tts_ready_delay** (*float*)
Play TTS in this much time if TTS `ready` message is not received
- **tts_params_enabled** (*bool*)
Enable TTS parameters?
- **tts_params_json** (*string*)
TTS parameters in JSON format
- **word_mappings** (*string*)
TTS Word Mappings (example: AI=A ay, 123=one two three)
- **emo_enabled** (*bool*)
Enable emotion in tts

The following are the control parameters for the emotion modifier

- **emotion** (*string*)
Supported emotions: `sad`, `happy`, `happy_tensed`, `afraid`
- **chunk_size** (*int*)

Chunk Size

- **semitones** (*float*)

The number of semitones to shift

- **gain** (*float*)

The gain at the Nyquist frequency

- **cutfreq** (*float*)

The filter's cutoff frequency in Hz

- **qfactor** (*float*)

The filter's width as a Q-factor

- **speed** (*float*)

Tremolo speed in Hz

- **depth** (*float*)

Tremolo depth as a percentage of the total amplitude

- **tempo** (*float*)

The ratio of new tempo to the old tempo

- **intensity** (*float*)

Gain adjustment in decibels (dB)

- **parameter_control** (*float*)

Parameter Control

The following parameters are experimental. Do not recommend to use.

- **enable_peer_chatbot** (*bool*)

Enable Peer Chatbot (alpha)

- **peer_chatbot_url** (*float*)

Chatbot URL on the Peer (alpha)

17.4 Animation API

This API describes commands to control the virtual animation rig for Sophia. It provides expressions, animations and visemes for Sophia. The actuators control needs to be set in animation mode in order to control actuators by this method.

The main animation pipeline and blending priorities:

Animation Layer	Pri-or-ity	Notes
Visemes	P1	Visemes has top priority. While visemes are active the other animations for lips are suppressed and may not be fully visible.
External animation	P2	External animation data could be streamed from mocap or other animation providers.
Animations	P3	Multiple animations can be blended with additive blending
Expressions	P4	One main expression can be set at one time. If one is triggered while other is active, the previous one would be blend out while new one is blending in

There are 3 types of keep alive animations: head motion, blinking and saccades. These can be managed with dynamic reconfigure parameters.

17.4.1 Topics Published

/hr/actuators/status

Message Type *hr_msgs/MotorStateList*

Description Provides feedback from actuators (where available, dynamixel motors only, others provide last sent position)

/hr/animation/current_state

Message Type *hr_msgs/pau*

Description Provides feedback about current state of animable components such as face expressions, head/neck orientations and joint angles

/hr/animation/fps

Message Type *std_msgs/UInt16*

Description Frames per second information

/hr/animation/queue_visemes

Message Type *hr_msgs/visemes*

Description List of visemes to play while talking

17.4.2 Topics Subscribed

/hr/animation/set_expression

Message Type *hr_msgs/SetExpression*

Description Set Expression with duration and magnitude

/hr/animation/set_animation

Message Type *hr_msgs/SetAnimation*

Description Set head animation

/hr/animation/set_arm_animation

Message Type *hr_msgs/SetAnimation*

Description Set arms animation

/hr/animation/set_face_target

Message Type *hr_msgs/Target*

Description Sets head and eyes to look at the point in space relative to the center of the head

/hr/animation/set_gaze_target

Message Type *hr_msgs/Target*

Description Sets eyes to look at the point in space relative to the center of the head

/hr/animation/set_head_tilt

Message Type *std_msgs/Float32*

Description Tilts the upper head by given angle

/hr/animation/set_neck_rotation

Message Type *geometry_msgs/Vector3*

Description Sets lower neck rotation by euler angles (in rad). x neck roll, y Neck Pitch

/hr/animation/set_state

Message Type *hr_msgs/TargetPosture*

Description Allows controlling animation with external source: i.e live mocap data. The posture should be set at high enough framerate(30 fps or more).

/hr/animation/set_animation_mode

Message Type *std_msgs/UInt8*

Description Allows setting of animation mode

/hr/animation/set_pau

Message Type *hr_msgs/pau*

Description Set current state of animable components such as face expressions, head/neck orientations and joint angles

17.4.3 External Animation Drivers

Name	Range
BrowOuterLeft	[-1; 1] - Down/Up
BrowInnerLeft	[-1; 1] - Down/Up
BrowCenter	[-1; 1] - Down/Up
BrowInnerRight	[-1; 1] - Down/Up
BrowOuterRight	[-1; 1] - Down/Up
UpperLidLeft	[-1; 1] - Eyes closed/ eyes open
UpperLidRight	[-1; 1] - Eyes closed/ eyes open
LowerLidLeft	[-1; 1] - Eyes closed/ eyes open
LowerLidRight	[-1; 1] - Eyes closed/ eyes open
CheekSquintLeft	[0; 1] - Neutral / Cheek up
CheekSquintRight	[0; 1] - Neutral / Cheek up
SneerLeft	[0; 1] - Neutral / Sneer

continues on next page

Table 2 – continued from previous page

Name	Range
SneerRight	[0; 1] - Neutral / Sneer
UpperLipLeft	[-1; 1] - Lip down / Lip up
UpperLipCenter	[-1; 1] - Lip down / Lip up
UpperLipRight	[-1; 1] - Lip down / Lip up
LowerLipLeft	[-1; 1] - Lip up / Lip down
LowerLipCenter	[-1; 1] - Lip up / Lip down
LowerLipRight	[-1; 1] - Lip up / Lip down
LipStretchLeft	[-1; 1] - Lip pucker / Lip wide
LipStretchRight	[-1; 1] - Lip pucker / Lip wide
FrownLeft	[0; 1] - Neutral / Frown
FrownRight	[0; 1] - Neutral / Frown
SmileLeft	[0; 1] - Neutral / Smile
SmileRight	[0; 1] - Neutral / Smile
Jaw	[0; 1] - Closed (neutral) / Open
EyesYaw	Angle in rad: Left/Right
EyesPitch	Angle in rad: Up/Down
HeadYaw	Angle in rad: Left/Right
HeadRoll	Angle in rad: Left/Right
HeadPitch	Angle in rad: Up/Down

17.4.4 Services

/hr/animation/available_expressions

Message Type *hr_msgs/StringArray*

Description List of all available expressions

/hr/animation/available_animations

Message Type *hr_msgs/StringArray*

Description List of all available head animations

/hr/animation/available_arm_animations

Message Type *hr_msgs/StringArray*

Description List of all available arm animations

/hr/animation/get_animation_length

Message Type *hr_msgs/GetAnimationLength*

Description Get length of the animation passed as argument during request

/hr/animation/get_arm_animation_length

Message Type *hr_msgs/GetAnimationLength*

Description Get length of the animation passed as argument during request

/hr/animation/get_arms_mode

Message Type *hr_msgs/GetMode*

Description Get value that corresponds to the current mode of the arms

/hr/animation/set_arms_mode

Message Type *hr_msgs/SetMode*

Description Set with value that corresponds to desired mode of the arms

/hr/animation/cancel_visemes

Message Type *std_srvs/Trigger*

Description Cancel the visemes queued up to play

17.4.5 Dynamic Reconfigure

/hr/animation/keepalive

Parameters

- **keepalive** (*bool*)
Enable head idle movement
- **blinking** (*bool*)
Enable blinking
- **blinking_interval_max** (*float*)
Set maximum blinking interval
- **blinking_interval_min** (*float*)
Set minimum blinking interval
- **eye_saccade** (*float*)
Enable eye saccade
- **eye_saccade_interval_max** (*float*)
Set maximum eye saccade interval
- **eye_saccade_interval_min** (*float*)
Set minimum eye saccade interval
- **eye_saccade_scale** (*float*)
Set eye saccade scale
- **blinking_interval_max** (*float*)
Set maximum blinking interval

/hr/animation/external_animation

Parameters

- **enabled** (*bool*)
Enable external_animation
- **orient_head** (*bool*)
Set Head Orientation as Neutral
- **set_neutral_face** (*bool*)
Neutralize face
- **reset_face** (*bool*)

-
- Reset Neutral face
 - **reset_head** (*bool*)
 - Reset head orientation

/hr/animation/parallax

Parameters

- **eye_distance** (*float*)
 - Set distance between eyes in cm
- **parallax_scale** (*float*)
 - Set Parallax Scale

17.5 Performance API

This API describes high level control commands to work with performances that were created in the timeline editor on the UI.

17.5.1 Topics Published

/hr/control/performances/events

Message Type *hr_msgs/Event*

Description Runs a specific performance event

/hr/control/performances/running_performance

Message Type *std_msgs/String*

Description Gets the currently running performance

17.5.2 Services

/hr/control/performances/run_by_name

Message Type *hr_msgs/RunByName*

Description Run performance by name provided. Name also includes path of folder structure separated by “/”

/hr/control/performances/pause

Message Type *hr_msgs/PerformanceCommand*

Description Pause existing performance.

/hr/control/performances/resume

Message Type *hr_msgs/PerformanceCommand*

Description Resume existing performance.

/hr/control/performances/stop

Message Type *hr_msgs/PerformanceCommand*

Description Stop current performance

/hr/control/performances/current

Message Type *hr_msgs/Current*

Description Complete details about currently running performance

/hr/control/performances/load_performance

Message Type *hr_msgs/Load*

Description Pass detail of performance to be loaded as argument and receive feedback of success/failure

/hr/control/performances/run_full_performance

Message Type *hr_msgs/RunByName*

Description Run the full performance by name without interruption

/hr/control/performances/reload_properties

Message Type *std_srvs/Trigger*

Description Reload properties of the performance

/hr/control/performances/set_properties

Message Type *hr_msgs/SetProperties*

Description Set properties of a specific performance identified by id

/hr/control/performances/unload

Message Type *std_srvs/Trigger*

Description Unload properties of the performance

17.5.3 Dynamic Reconfigure

/hr/control/speech_motions

Parameters

- **automated_arm_gestures** (*bool*)
Perform automated arm gestures?
- **da_rules** (*string*)
Set the DA Rules
- **enable** (*bool*)
Enable keyword rules
- **global_probability** (*float*)
Adjust all probabilities (-1 no rules applied, 1 all rules, 0 Unchanged)
- **interactive** (*bool*)
Only in interactive mode?
- **keyword_rules** (*string*)
Set the keyword rules

/hr/control/performances

Parameters

- **autopause** (*bool*)

Enable auto pause for performances (between separate timelines)

MESSAGE DEFINITION

18.1 Messages

18.1.1 hr_msgs/AudioStream

Message Field	Field Type	Description
header	Header	http://docs.ros.org/en/melodic/api/std_msgs/html/msg/Header.html
encoding	uint8	
SINT_8_PCM	uint8	
UINT_8_PCM	uint8	
SINT_16_PCM	uint8	
SINT_24_PCM	uint8	
SINT_32_PCM	uint8	
FLOAT_32	uint8	
FLOAT_64	uint8	
is_bigendian	uint8	
channels	uint8	
sample_rate	uint32	
data	uint8[]	

18.1.2 hr_msgs/ChatMessage

Message Field	Field Type	Description
utterance	string	text of utterance
lang	string	language code of the utterance
confidence	uint8	the confidence of the text (0-100)
source	string	source of the utterance
audio_path	string	audio path of the speech that gives the utterance

18.1.3 hr_msgs/ChatResponse

Message Field	Field Type	Description
text	String	Response given by chatbot
lang	String	Language
label	String	Label
request_id	String	Request id (used internally)
agent_id	String	Agent id (used internally)
sid	String	Session id (used internally)

18.1.4 hr_msgs/ChatResponses

Message Field	Field Type	Description
responses	ChatResponse[]	Responses given by chatbot

18.1.5 hr_msgs/Event

Message Field	Field Type	Description
event	String	Event name and/or type
time	float64	Time at which event occurs

18.1.6 hr_msgs/MotorState

Message Field	Field Type	Description
timestamp	float64	motor state is at this time
name	string	motor name
position	float64	current position (in r/s)
speed	float64	current speed (in r/s)
load	float64	current load - ratio of applied torque over maximum torque
voltage	float64	current voltage (V)
temperature	int32	current temperature (degrees Celsius)
torque_enabled	bool	indicates enabled torque on motor
torque_limit	float64	current limit applied on motor (0-1)
errorCode	uint8	last error codes

18.1.7 hr_msgs/MotorStateList

Message Field	Field Type	Description
motor_states	hr_msgs/MotorState[]	array of motor states

18.1.8 hr_msgs/MotorValues

Message Field	Field Type	Description
names	string[]	Joint/actuator names
values	float64[]	Corresponding values: For torque: 0-1 (1 - max torque) For speed (rad/s)

18.1.9 hr_msgs/MotorValuesBool

Message Field	Field Type	Description
names	string[]	Joint/actuator name
values	bool[]	Values that are based on the service: For torque: True if torque is enabled.

18.1.10 hr_msgs/pau

Message Field	Field Type	Description
m_headRotation	geometry_msgs/Quaternion	Orientation of head
m_headTranslation	geometry_msgs/Vector3	Position of head
m_neckRotation	geometry_msgs/Quaternion	Orientation of neck
m_eyeGazeLeftPitch	float32	Pitch of left eye
m_eyeGazeLeftYaw	float32	Yaw of left eye
m_eyeGazeRightPitch	float32	Pitch of right eye
m_eyeGazeRightYaw	float32	Yaw of right eye
m_coeffs	float32[]	Array of blendshape coefficients that describe the expression of a virtual face
m_shapekeys	string[]	Setting shapekey values; names must be passed
m_joints	float32[]	Array of arm/hand joint angles
m_joints	string[]	Array of arm/joint names

18.1.11 hr_msgs/SensorData

Message Field	Field Type	Description
names	string[]	SensorNames
values	float64[]	Raw Sensor Values

18.1.12 hr_msgs/SetAnimation

Message Field	Field Type	Description
name	string	name of the animation
repeat	int32	how many times to repeat
speed	float32	speed of the animation
magnitude	float32	magnitude of the animation

18.1.13 hr_msgs/SetExpression

Message Field	Field Type	Description
name	string	name of the expression
magnitude	float32	magnitude of the expression
duration	duration	duration of the expression (in seconds)

18.1.14 hr_msgs/State

Message Field	Field Type	Description
ts	time	
faces	Face[]	
poses	Face[]	
salientpoints	SalientPoint[]	

18.1.15 hr_msgs/Target

Message Field	Field Type	Description
x	float32	x coordinate
y	float32	Y coordinate
z	float32	z coordinate
speed	float32 MotorState	motion speed(0-1)

18.1.16 hr_msgs/TargetPosture

Message Field	Field Type	Description
names	string[]	Joint/actuator names
values	float64[]	Corresponding angles of actuators in rad. In the min-max range from ROS param.

18.1.17 hr_msgs/TTS

Message Field	Field Type	Description
utterance	string	text of utterance
lang	string	language code of the utterance
request_id	string	for internal use
agent_id	string	for internal use
sid	string	for internal use

18.1.18 hr_msgs/VADMessage

Message Field	Field Type	Description
decibel	float32	decibel of the VAD audio chunk
frequency	float32	frequency of the VAD audio chunk
duration	float32	duration of the VAD audio chunk (sec)

18.1.19 hr_msgs/viseme

Message Field	Field Type	Description
name	string	Name
start	time	Starting time
rampin	float32	Time to ramp in in % of duration
rampout	float32	Time to ramp out in % of duration
magnitude	float32	Magnitude in blender

18.1.20 hr_msgs/visemes

Message Field	Field Type	Description
data	Viseme[]	

Refer to *hr_msgs/viseme*

18.2 Services

18.2.1 hr_msgs/AgentFeedback

Service Type	Message Field	Field Type	Description
Resquest	request_id	string	chat request id
	hybrid	bool	if it is in hybrid mode
	chosen	bool	whether this agent is chosen
Response	success	bool	True if succeed otherwise False

18.2.2 hr_msgs/AgentRegister

Service Type	Message Field	Field Type	Description
Resquest	node	string	unique chat agent name
	level	int32	default: 250 The agents with the same level will be executed in the same batch [100-200): before_builtin_agents, [200-300): among_builtin_agents, [300-400): after_builtin_agents.
	level_priority	int32	reserved
	weight	float64	score factor 0-1
	languages	string[]	language codes. Empty means for all languages
Response	success	bool	True if succeed otherwise False
	message	string	error message

18.2.3 hr_msgs/AgentUnregister

Service Type	Message Field	Field Type	Description
Resquest	node	string	unique chat agent name
Response	success	bool	True if succeed otherwise False
	message	string	error message

18.2.4 hr_msgs/ChatService

Service Type	Message Field	Field Type	Description
Request	text	string	text of chat message
	session	string	session id
	lang	string	language code of the chat message
	context	string	context of the chat message (not used)
Response	response	string	the response from chat server

18.2.5 hr_msgs/ChatSession

Service Type	Message Field	Field Type	Description
Request	session	string	session id to initialize (can be empty)
Response	session	string	session id initialized

18.2.6 hr_msgs/Current

Service Type	Message Field	Field Type	Description
Request			
Response	performance	string	full breakdown of currently running performance
	current_time	float32	running time of the performance
	running	bool	currently running?
	paused	bool	currently paused?

18.2.7 hr_msgs/GetMode

Service Type	Message Field	Field Type	Description
Request			
Response	value	uint16	Arms mode identifier

18.2.8 hr_msgs/GetIntent

Service Type	Message Field	Field Type	Description
Request	text	string	the name of the scene
	lang	string	the language code (BCP 47)
Response	intent	string	the intent label of the text
	confidence	float64	the confidence level of the detected intent

18.2.9 hr_msgs/GetAnimationLength

Service Type	Message Field	Field Type	Description
Request	animation	string	animation identifier
Response	length	float32	length

18.2.10 hr_msgs/Load

Service Type	Message Field	Field Type	Description
Request	id	string	Identifier of performance to be loaded
Response	bool	success	Load Success/Failure?
	performance	string	Details of performance loaded

18.2.11 hr_msgs/LoadPerformance

Service Type	Message Field	Field Type	Description
Request	performance	string	Details of performance loaded
Response	bool	success	Load Success/Failure?

18.2.12 hr_msgs/PerformanceCommand

Service Type	Message Field	Field Type	Description
Response	success	bool	
	time	float64	

18.2.13 hr_msgs/RunByName

Service Type	Message Field	Field Type	Description
Request	id	string	Performancepath and name folders and name are separated by “/”.
Response	success	bool	Returns True if success

18.2.14 hr_msgs/SetMode

Service Type	Message Field	Field Type	Description
Request	mode	uint16	Arms mode identifier
Response	result	bool	Result of setting process

18.2.15 hr_msgs/SetProperties

Service Type	Message Field	Field Type	Description
Request	id	string	Identifier of performance whose properties are to be set
	properties	string	Properties to be set
Response	success	bool	Feedback about success or failure

18.2.16 hr_msgs/SetScene

Service Type	Message Field	Field Type	Description
Request	scene	string	the name of the scene
Response	success	bool	true if scene is set successfully, false otherwise
	message	string	some extra message information about this call

18.2.17 hr_msgs/SetActuatorControl

Service Type	Message Field	Field Type	Description
Request	Control	int32	CONTROL_DISABLE(0): disable motor CONTROL_MANUAL(1): control motor directly CONTROL_ANIMATION(2): control by animation
	actuators	string[]	name of motion
Response	success	bool	indicates if the request was successful
	error	string	error messages with reason why it was not successful

18.2.18 hr_msgs/StringTrigger

Service Type	Message Field	Field Type	Description
Request	data	string	trigger message
Response	message	string	informational, e.g. for error messages
	success	string	indicate successful run of triggered service

18.2.19 hr_msgs/StringArray

Service Type	Message Field	Field Type	Description
Request			empty
Response	data	string[]	data array

18.2.20 hr_msgs/TTSDData

Service Type	Message Field	Field Type	Description
Resquest	txt	string	the text
	lang	string	language code of the text
Response	data	float32	The audio generated by tts

18.2.21 hr_msgs/TTSLength

Service Type	Message Field	Field Type	Description
Resquest	txt	string	the text
	lang	string	language code of the text
Response	length	float32	length of the audio generated by tts (in seconds)

SDK TROUBLESHOOTING

Issue

```
usage: aws [options] <command> <subcommand> [<subcommand> ...]
* * *
aws: error: argument operation: Invalid choice, valid choices are:
* * *
```

Solution Could be incompatible awscli. Upgrade awscli (version 1) to 1.18 or above.

Issue

```
fatal error: An error occurred (AccessDenied) when calling the
ListObjectsV2 operation: Access Denied
```

Solution The credentials are not set correctly. Check the steps in *SDK Setup*.

Issue

```
ERROR: The Compose file '/opt/hansonrobotics/hrsdk/compose/control.yaml'
is invalid because:
Unsupported config option for services.control: 'runtime'
```

Solution Check the version of docker-compose. It needs to be no older than 1.25. See *SDK Setup*

Issue

```
Remote error from secret service: org.freedesktop.Secret.Error.IsLocked:
Cannot create an item in a locked collection
Error saving credentials: error storing credentials - err: exit status 1,
out: `Cannot create an item in a locked collection`
```

or

```
Error saving credentials: error storing credentials - err: exit status 1,
out: `Cannot autolaunch D-Bus without X11 $DISPLAY`
```

Solution Uninstall the package golang-docker-credential-helpers : <https://github.com/docker/compose/issues/6023>

Issue

```
mkdir: cannot create directory '/tmp/hrsdk.lock': File exists  
Could not create lock directory '/tmp/hrsdk.lock'. The process may have started
```

Solution Run hrsdk stop and try again

Issue

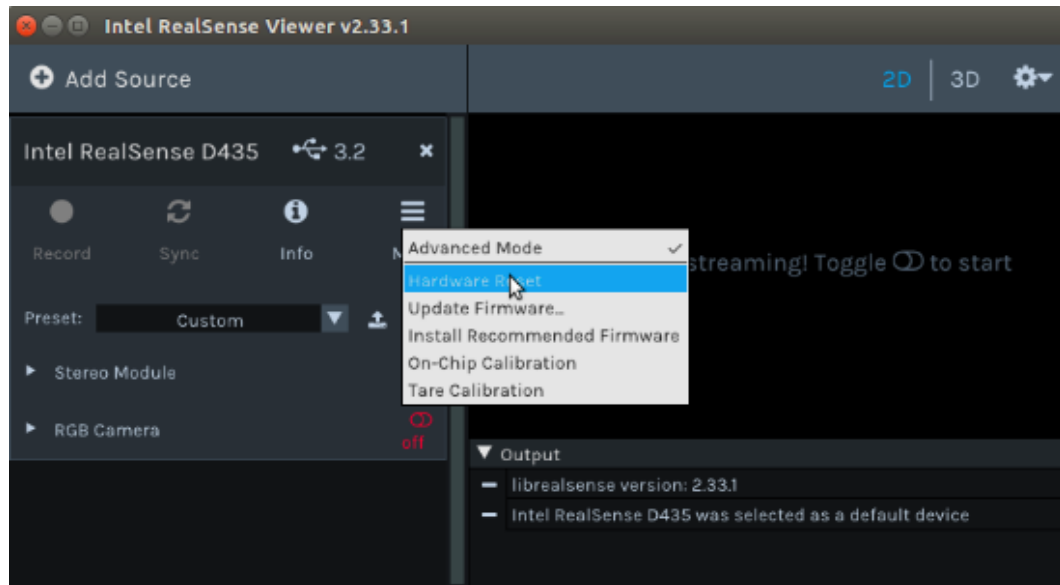
```
touch: setting times of '/tmp/empty.blend': Permission denied
```

Solution Remove the folder: `sudo rm -r /tmp/empty.blend`

Issue

```
Realsense camera disconnected
```

Solution This is suspected to be linked to the bug in camera firmware. To solve the issue one can stop the acquisition system (terminate all nodes), and go through the Hardware Reset procedure using the realsense-viewer. The system usually starts properly after doing this. [\[Source\]](#)



Issue

```
docker.errors.NullResource: Resource ID was not provided  
[18328] Failed to execute script docker-compose
```

Solution Run hrsdk pull to download the missing containers

Issue

```
Error response from daemon: pull access denied for ...  
222132024866.dkr.ecr.ap-east-1.amazonaws.com/hansonrobotics/***, ...  
repository does not exist or may require 'docker login': ...  
denied: Your authorization token has expired. Reauthenticate and try again.
```

Solution The docker authentication token expired. Try to run `hrsdk login` to refresh the token.

ROBOT TROUBLESHOOTING

20.1 Motor

Issue

Motor is not moving

Solution Make sure the power of robot is on (either DC/AC power). Check if the motor switch on the side of robot/soft switch on the back is on or not. Make sure all USB is plugged in NUC and try to restart the robot

Issue

Motor and finger are moving but the finger doesn't fold as other fingers

Solution Make sure all the pin on finger is not loose and in correct position

Issue

Elbow pitch motor is overload

Solution Make sure the offset position of the arm is correct, both arms should be in similar height under sitting pose. Please try to apply a bit force to reset it into correct position. If it is hard to reset, do not force it too much, call Hanson Robotics Support team for next step.

Warning: Do not force the elbow too much or the gear inside would be broken

20.2 Perception/Audio

Issue

Robot not able to hear what we said

Solution Make sure the internet is on and make sure the speech recognition is on. Check the USB of the microphone is connected to the NUC

Issue

Robot not tracking the face of the people

Solution Make sure the robot is online, and robot would only do the face tracking in interactive mode. Set the correct tracking target in the setting page, e.g. Looking to nearest face

Issue

Robot not able speaking Chinese

Solution Make sure the robot is online and the chatbot/speech is select the CN mode

Issue

Robot is not speaking

Solution Make sure the audio volume is on and the chat is enabled

INDICES AND TABLES

- `genindex`
- `search`

Symbols

/hr/actuators/dynamixel/motor_states	configuration value, 45
/hr/actuators/dynamixel/set_actuator_speeds	configuration value, 47
/hr/actuators/dynamixel/set_torque_enable	configuration value, 47
/hr/actuators/dynamixel/set_torque_limit	configuration value, 47
/hr/actuators/lhand/pose	configuration value, 45
/hr/actuators/outer/pose	configuration value, 46
/hr/actuators/pose	configuration value, 45
/hr/actuators/rhand/pose	configuration value, 45
/hr/actuators/set_control	configuration value, 47
/hr/actuators/status	configuration value, 53
/hr/actuators/top/pose	configuration value, 46
/hr/animation/available_animations	configuration value, 55
/hr/animation/available_arm_animations	configuration value, 55
/hr/animation/available_expressions	configuration value, 55
/hr/animation/cancel_visemes	configuration value, 56
/hr/animation/current_state	configuration value, 53
/hr/animation/external_animation	configuration value, 56
/hr/animation/fps	configuration value, 53
/hr/animation/get_animation_length	configuration value, 55
/hr/animation/get_arm_animation_length	configuration value, 55
/hr/animation/get_arms_mode	configuration value, 55
/hr/animation/keepalive	configuration value, 56
/hr/animation/parallax	configuration value, 57
/hr/animation/queue_visemes	configuration value, 53
/hr/animation/set_animation	configuration value, 53
/hr/animation/set_animation_mode	configuration value, 54
/hr/animation/set_arm_animation	configuration value, 53
/hr/animation/set_arms_mode	configuration value, 55
/hr/animation/set_expression	configuration value, 53
/hr/animation/set_face_target	configuration value, 54
/hr/animation/set_gaze_target	configuration value, 54
/hr/animation/set_head_tilt	configuration value, 54
/hr/animation/set_neck_rotation	configuration value, 54
/hr/animation/set_pau	configuration value, 54
/hr/animation/set_state	configuration value, 54
/hr/behavior/interactive_fiction/cancel_tree	configuration value, 39
/hr/behavior/interactive_fiction/reload_trees	configuration value, 39
/hr/behavior/interactive_fiction/start_tree	configuration value, 39
/hr/control/arms	configuration value, 49
/hr/control/audio/stream	configuration value, 49
/hr/control/performance_generation	configuration value, 49
/hr/control/performances	

configuration value, 58	configuration value, 37
/hr/control/performances/current	/hr/interaction/chatbot_input
configuration value, 58	configuration value, 36
/hr/control/performances/events	/hr/interaction/chatbot_response
configuration value, 57	configuration value, 36
/hr/control/performances/load_performance	/hr/interaction/chatbot_responses
configuration value, 58	configuration value, 36
/hr/control/performances/pause	/hr/interaction/get_context
configuration value, 57	configuration value, 37
/hr/control/performances/reload_properties	/hr/interaction/get_intent
configuration value, 58	configuration value, 37
/hr/control/performances/resume	/hr/interaction/register
configuration value, 57	configuration value, 37
/hr/control/performances/run_by_name	/hr/interaction/run_form
configuration value, 57	configuration value, 37
/hr/control/performances/run_full_performance	/hr/interaction/session
configuration value, 58	configuration value, 37
/hr/control/performances/running_performance	/hr/interaction/set_scene
configuration value, 57	configuration value, 37
/hr/control/performances/set_properties	/hr/interaction/unregister
configuration value, 58	configuration value, 37
/hr/control/performances/stop	/hr/perception/<microphone name>
configuration value, 57	configuration value, 34
/hr/control/performances/unload	/hr/perception/face_markers
configuration value, 58	configuration value, 33
/hr/control/speech/data (<i>srv</i>)	/hr/perception/hand_events
configuration value, 50	configuration value, 33
/hr/control/speech/data (<i>topic</i>)	/hr/perception/hear/event
configuration value, 50	configuration value, 33
/hr/control/speech/event	/hr/perception/hear/interim_speech
configuration value, 50	configuration value, 33
/hr/control/speech/interrupt	/hr/perception/hear/sentence
configuration value, 50	configuration value, 33
/hr/control/speech/length	/hr/perception/hear/words
configuration value, 50	configuration value, 33
/hr/control/speech/say	/hr/perception/joint_states
configuration value, 50	configuration value, 33
/hr/control/speech/tts_control	/hr/perception/poses
configuration value, 50	configuration value, 33
/hr/control/speech/tts_stream	/hr/perception/state
configuration value, 50	configuration value, 34
/hr/control/speech/tts_talker	/hr/perception/ui_image
configuration value, 51	configuration value, 33
/hr/control/speech_motions	/hr/perception/vad
configuration value, 58	configuration value, 33
/hr/interaction/available_agents	/hr/sensors/audio/<microphone name>
configuration value, 36	configuration value, 31
/hr/interaction/available_forms	/hr/sensors/left_hand
configuration value, 36	configuration value, 31
/hr/interaction/available_scenes	/hr/sensors/right_hand
configuration value, 36	configuration value, 31
/hr/interaction/chat	/hr/sensors/vision/chest_cam/usb_cam/image_raw
configuration value, 36	configuration value, 32
/hr/interaction/chatbot	/hr/sensors/vision/eye_cam/usb_cam/image_raw

configuration value, 32
 /hr/sensors/vision/realsense/camera/accel/sample
 configuration value, 31
 /hr/sensors/vision/realsense/camera/aligned_depth_
 configuration value, 32
 /hr/sensors/vision/realsense/camera/aligned_depth_
 configuration value, 32
 /hr/sensors/vision/realsense/camera/color/image_raw
 configuration value, 32
 /hr/sensors/vision/realsense/camera/gyro/sample
 configuration value, 31

A

AlwaysFail
 configuration value, 41
 AlwaysSuccess
 configuration value, 41
 Animation
 configuration value, 43

C

Chat
 configuration value, 41
 Chatbot Enable
 configuration value, 42
 Check param
 configuration value, 43
 Check Time
 configuration value, 42
 CheckSpeech
 configuration value, 43
 configuration value
 /hr/actuators/dynamixel/motor_states, 45
 /hr/actuators/dynamixel/set_actuator_speeds,
 47
 /hr/actuators/dynamixel/set_torque_enable,
 47
 /hr/actuators/dynamixel/set_torque_limit,
 47
 /hr/actuators/lhand/pose, 45
 /hr/actuators/outer/pose, 46
 /hr/actuators/pose, 45
 /hr/actuators/rhand/pose, 45
 /hr/actuators/set_control, 47
 /hr/actuators/status, 53
 /hr/actuators/top/pose, 46
 /hr/animation/available_animations, 55
 /hr/animation/available_arm_animations,
 55
 /hr/animation/available_expressions, 55
 /hr/animation/cancel_visemes, 56
 /hr/animation/current_state, 53
 /hr/animation/external_animation, 56
 /hr/animation/fps, 53
 /hr/animation/get_animation_length, 55
 /hr/animation/get_arm_animation_length,
 55
 /hr/animation/get_arms_mode, 55
 /hr/animation/keepalive, 56
 /hr/animation/keepalive, 56
 /hr/animation/queue_visemes, 53
 /hr/animation/set_animation, 53
 /hr/animation/set_animation_mode, 54
 /hr/animation/set_arm_animation, 53
 /hr/animation/set_arms_mode, 55
 /hr/animation/set_expression, 53
 /hr/animation/set_face_target, 54
 /hr/animation/set_gaze_target, 54
 /hr/animation/set_head_tilt, 54
 /hr/animation/set_neck_rotation, 54
 /hr/animation/set_pau, 54
 /hr/animation/set_state, 54
 /hr/behavior/interactive_fiction/cancel_tree,
 39
 /hr/behavior/interactive_fiction/reload_trees,
 39
 /hr/behavior/interactive_fiction/start_tree,
 39
 /hr/control/arms, 49
 /hr/control/audio/stream, 49
 /hr/control/performance_generation, 49
 /hr/control/performances, 58
 /hr/control/performances/current, 58
 /hr/control/performances/events, 57
 /hr/control/performances/load_performance,
 58
 /hr/control/performances/pause, 57
 /hr/control/performances/reload_properties,
 58
 /hr/control/performances/resume, 57
 /hr/control/performances/run_by_name, 57
 /hr/control/performances/run_full_performance,
 58
 /hr/control/performances/running_performance,
 57
 /hr/control/performances/set_properties,
 58
 /hr/control/performances/stop, 57
 /hr/control/performances/unload, 58
 /hr/control/speech/data (srv), 50
 /hr/control/speech/data (topic), 50
 /hr/control/speech/event, 50
 /hr/control/speech/interrupt, 50
 /hr/control/speech/length, 50
 /hr/control/speech/say, 50
 /hr/control/speech/tts_control, 50
 /hr/control/speech/tts_stream, 50
 /hr/control/speech/tts_talker, 51

- /hr/control/speech_motions, 58
- /hr/interaction/available_agents, 36
- /hr/interaction/available_forms, 36
- /hr/interaction/available_scenes, 36
- /hr/interaction/chat, 36
- /hr/interaction/chatbot, 37
- /hr/interaction/chatbot_input, 36
- /hr/interaction/chatbot_response, 36
- /hr/interaction/chatbot_responses, 36
- /hr/interaction/get_context, 37
- /hr/interaction/get_intent, 37
- /hr/interaction/register, 37
- /hr/interaction/run_form, 37
- /hr/interaction/session, 37
- /hr/interaction/set_scene, 37
- /hr/interaction/unregister, 37
- /hr/perception/<microphone name>, 34
- /hr/perception/face_markers, 33
- /hr/perception/hand_events, 33
- /hr/perception/hear/event, 33
- /hr/perception/hear/interim_speech, 33
- /hr/perception/hear/sentence, 33
- /hr/perception/hear/words, 33
- /hr/perception/joint_states, 33
- /hr/perception/poses, 33
- /hr/perception/state, 34
- /hr/perception/ui_image, 33
- /hr/perception/vad, 33
- /hr/sensors/audio/<microphone name>, 31
- /hr/sensors/left_hand, 31
- /hr/sensors/right_hand, 31
- /hr/sensors/vision/chest_cam/usb_cam/image_raw, 32
- /hr/sensors/vision/eye_cam/usb_cam/image_raw, 32
- /hr/sensors/vision/realsense/camera/accel/sample, 31
- /hr/sensors/vision/realsense/camera/aligned_depth_to_color/image_raw, 32
- /hr/sensors/vision/realsense/camera/aligned_depth_to_infra1/image_raw, 32
- /hr/sensors/vision/realsense/camera/color/image_raw, 32
- /hr/sensors/vision/realsense/camera/gyro/sample, 31
- AlwaysFail, 41
- AlwaysSuccess, 41
- Animation, 43
- Chat, 41
- Chatbot Enable, 42
- Check param, 43
- Check Time, 42
- CheckSpeech, 43
- Decision, 40
 - configuration value, 40
- Idle, 43
 - configuration value, 43
- In Last Heard Sentence
 - configuration value, 41
- Inverter
 - configuration value, 41
- Is Listening
 - configuration value, 44
- Is Talking
 - configuration value, 44
- Max time (ms)
 - configuration value, 41
- Move To
 - configuration value, 42
- Parallel
 - configuration value, 40
- People Detected
 - configuration value, 44
- Person
 - configuration value, 44
- Priority, 40
- Repeat, 41
- Repeat Until Failure, 41
- Repeat Until Success, 41
- Say, 42
- Sequence, 40
- Set Dynamic Param, 42
- Set Dynamic Params, 42
- Timeline, 43
- Trigger Event, 42
- Update Param, 44
- Wait (ms), 41
- Wait For, 42

D

- Decision
 - configuration value, 40

I

- Idle
 - configuration value, 43
- In Last Heard Sentence
 - configuration value, 41
- Inverter
 - configuration value, 41
- Is Listening
 - configuration value, 44
- Is Talking
 - configuration value, 44

M

- Max time (ms)
 - configuration value, 41
- Move To
 - configuration value, 42

P

- Parallel
 - configuration value, 40
- People Detected
 - configuration value, 44
- Person
 - configuration value, 44

Priority
 configuration value, [40](#)

R

Repeat
 configuration value, [41](#)
Repeat Until Failure
 configuration value, [41](#)
Repeat Until Success
 configuration value, [41](#)

S

Say
 configuration value, [42](#)
Sequence
 configuration value, [40](#)
Set Dynamic Param
 configuration value, [42](#)
Set Dynamic Params
 configuration value, [42](#)

T

Timeline
 configuration value, [43](#)
Trigger Event
 configuration value, [42](#)

U

Update Param
 configuration value, [44](#)

W

Wait (*ms*)
 configuration value, [41](#)
Wait For
 configuration value, [42](#)